UNIVERSIDADE ESTADUAL DO MARANHÃO
CENTRO DE CIÊNCIAS TECNOLÓGICAS
NÚCLEO DE TECNOLOGIAS PARA EDUCAÇÃO
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ANDRÉ PEREIRA FERREIRA

ANÁLISE COMPARATIVA DE DESEMPENHO EM AMBIENTES DE PERSISTÊNCIA EM MEMÓRIA: o impacto da arquitetura Worker do FrankenPHP no framework CodeIgniter 4 frente ao ecossistema Laravel Octane

Grajaú - MA
2026

ANDRÉ PEREIRA FERREIRA

ANÁLISE COMPARATIVA DE DESEMPENHO EM AMBIENTES DE PERSISTÊNCIA EM MEMÓRIA: o impacto da arquitetura Worker do FrankenPHP no framework CodeIgniter 4 frente ao ecossistema Laravel Octane

Projeto de pesquisa apresentado ao curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas como requisito da disciplina de TCC, para fins de planejamento, organização e elaboração de trabalhos científicos.

Orientadora: Titulação e nome.

Grajaú - MA

2026

SUMÁRIO

1 INTRODUÇÃO.....6
2 OBJETIVOS.....8
2.1 Objetivo Geral.....8
2.2 Objetivos Específicos.....8
3 REFERENCIAL TEÓRICO.....9
3.1 A Arquitetura Tradicional do PHP: Modelo Request-Response-Die e o PHP-FPM.....9
3.2 Servidores de Aplicação e Persistência em Memória no PHP.....9
3.3 O Ecossistema FrankenPHP e a Arquitetura Worker.....10
3.4 Avaliação de Desempenho e Benchmarking de Sistemas Web.....10
4 METODOLOGIA.....11
4.1 Caracterização da pesquisa.....11
4.2 Ambiente experimental.....11
4.3 Instrumentação e geração de carga.....12
4.4 Variáveis e métricas analisadas.....12
4.5 Procedimentos de análise.....13
5 CRONOGRAMA.....14
REFERÊNCIAS.....15

1 INTRODUÇÃO

A evolução das aplicações web tem ampliado a necessidade de arquiteturas capazes de oferecer maior desempenho, escalabilidade e menor tempo de resposta diante do crescimento da demanda por serviços digitais. Nesse contexto, as decisões relacionadas ao modelo de execução das aplicações assumem papel relevante, especialmente em sistemas desenvolvidos com a linguagem PHP, amplamente utilizada no desenvolvimento de aplicações web. Tradicionalmente, aplicações PHP são executadas seguindo um modelo no qual cada requisição HTTP é processada de forma independente, sendo comum a utilização do PHP-FPM (FastCGI Process Manager) para gerenciamento dos processos responsáveis pela execução da aplicação. Embora esse modelo apresente ampla maturidade e utilização em ambientes de produção, parte do custo computacional associado à inicialização do framework e à preparação do ambiente de execução ocorre repetidamente a cada requisição.

Com o avanço dos servidores de aplicação modernos, novas possibilidades de execução de aplicações PHP passaram a explorar a permanência da aplicação em memória. Tecnologias como Swoole e FrankenPHP possibilitam modelos de execução baseados em processos persistentes, reduzindo a necessidade de repetir etapas de inicialização da aplicação a cada requisição. No ecossistema Laravel, essa abordagem é utilizada pelo Laravel Octane, que permite a execução da aplicação sobre servidores de aplicação de longa duração. Dessa forma, torna-se relevante investigar se os benefícios proporcionados pela persistência em memória também podem ser obtidos em outros frameworks PHP, especialmente em aplicações desenvolvidas com CodeIgniter 4.

O CodeIgniter 4 (CI4) caracteriza-se por ser um framework PHP de estrutura relativamente leve e utilizado no desenvolvimento de aplicações web. Entretanto, diferentemente do ecossistema Laravel com Octane, o CI4 não oferece suporte nativo ao modo Worker do FrankenPHP. Essa limitação cria uma questão relevante para aplicações existentes que utilizam o framework: seria possível obter ganhos significativos de desempenho por meio da adoção do FrankenPHP em modo Worker, mesmo com a necessidade de mecanismos adicionais para manter a aplicação em memória? Além disso, é necessário verificar se a utilização do FrankenPHP em seu modo Clássico apresenta vantagens suficientes em relação ao modelo tradicional baseado em PHP-FPM para justificar sua adoção.

Diante desse cenário, esta pesquisa será direcionada à análise comparativa do desempenho do CodeIgniter 4 submetido a diferentes arquiteturas de execução. Serão considerados o modelo tradicional baseado em PHP-FPM, o FrankenPHP em modo Clássico e o FrankenPHP em modo Worker, sendo este último implementado mediante uma solução de integração que possibilite a permanência da aplicação em memória. Como referência adicional, será utilizado o Laravel com Laravel Octane, permitindo estabelecer um parâmetro de comparação com uma solução consolidada no ecossistema PHP.

A relevância da pesquisa está relacionada à necessidade de obtenção de dados experimentais que auxiliem na tomada de decisões sobre modernização e escalabilidade de aplicações PHP. A identificação de possíveis ganhos de desempenho poderá contribuir tanto para novos projetos quanto para aplicações existentes que utilizam CodeIgniter 4 e que necessitam aumentar sua capacidade de processamento sem necessariamente realizar uma migração completa para outro framework ou linguagem. Além disso, a avaliação do consumo de recursos poderá contribuir para a análise da relação entre desempenho e utilização de infraestrutura computacional.

Para realizar essa investigação, será conduzido um estudo experimental e quantitativo, utilizando ambientes controlados e equivalentes em termos de recursos computacionais. As aplicações serão executadas em contêineres Docker e submetidas a testes de carga utilizando o Grafana k6, enquanto métricas relacionadas ao desempenho e ao consumo de recursos serão coletadas para posterior análise. Serão consideradas, principalmente, métricas de vazão, expressa em requisições por segundo (RPS), latência e utilização de recursos computacionais.

Assim, a pesquisa buscará produzir evidências experimentais sobre o impacto da persistência em memória no desempenho de aplicações desenvolvidas com CodeIgniter 4, contribuindo para a compreensão das possibilidades de utilização do FrankenPHP em diferentes modos de operação e estabelecendo uma comparação com arquiteturas tradicionais e com uma solução consolidada no ecossistema Laravel.

2 OBJETIVOS

2.1 Objetivo Geral

Analisar comparativamente o desempenho do framework CodeIgniter 4 quando executado sob diferentes arquiteturas de servidores de aplicação PHP, com ênfase nos efeitos da persistência em memória proporcionada pelo FrankenPHP em modo Worker, utilizando o PHP-FPM e o FrankenPHP em modo Clássico como cenários de comparação e o Laravel Octane como referência de desempenho.

2.2 Objetivos Específicos

- Descrever as características dos modelos de execução PHP-FPM, FrankenPHP em modo Clássico e FrankenPHP em modo Worker, considerando seus mecanismos de processamento e gerenciamento da aplicação em memória;
- Implementar ambientes experimentais controlados para a execução do CodeIgniter 4 nas diferentes arquiteturas analisadas, mantendo equivalência nas condições de hardware e infraestrutura;
- Avaliar o desempenho das arquiteturas por meio de testes de carga, considerando métricas como vazão, expressa em requisições por segundo (RPS), e latência;
- Comparar os resultados obtidos pelo CodeIgniter 4 com PHP-FPM, FrankenPHP Clássico e FrankenPHP Worker, utilizando o Laravel Octane como referência adicional de desempenho;
- Analisar a relação entre os ganhos de desempenho observados, o consumo de recursos computacionais e a complexidade necessária para adoção de arquiteturas baseadas em persistência de memória.

3 REFERENCIAL TEÓRICO

3.1 A Arquitetura Tradicional do PHP: Modelo Request-Response-Die e o PHP-FPM

A arquitetura histórica de execução da linguagem PHP fundamenta-se no modelo Share-Nothing (sem compartilhamento de estado), no qual cada requisição HTTP recebida pelo servidor resulta na inicialização do interpretador, carregamento dos arquivos da aplicação, execução das instruções e imediata destruição de todos os recursos da memória ao término da resposta (LOCKHART, 2015).

Com a consolidação das aplicações web modernas e a necessidade de otimização de infraestrutura, o gerenciamento de processos evoluiu com a adoção do PHP-FPM (FastCGI Process Manager) (PHP DOCUMENTATION GROUP, 2024). O PHP-FPM atua mantendo pools de processos pré-inicializados para servir requisições sem o custo de invocar o binário do PHP a cada chamada (PHP DOCUMENTATION GROUP, 2024). No entanto, embora reduza a sobrecarga do sistema operacional, o modelo tradicional sob PHP-FPM ainda impõe que frameworks de grande porte realizem o ciclo completo de bootstrapping (carregamento de rotas, dependências e configurações) a cada nova requisição (LOCKHART, 2015).

3.2 Servidores de Aplicação e Persistência em Memória no PHP

Para superar os gargalos associados ao bootstrapping repetitivo, surgiram alternativas baseadas em loops de eventos e servidores de aplicação de longa duração. Nesse paradigma, a aplicação é inicializada uma única vez e mantida residente na memória RAM, permanecendo pronta para processar múltiplas requisições concorrentes sem recomputar a fase de boot.

No ecossistema Laravel, essa abordagem foi padronizada através do Laravel Octane (OTWELL, 2024). O Octane serve como uma camada de abstração que gerencia a persistência em memória e lida nativamente com os desafios decorrentes desse modelo, como o vazamento de memória (memory leaks) e a poluição do estado global entre requisições de usuários distintos (OTWELL, 2024).

Por outro lado, frameworks mais leves e tradicionais, como o CodeIgniter 4 (CI4), não contam com suporte nativo ou ecossistema integrado para execução em loops persistentes, sendo tradicionalmente projetados para o ciclo de vida dinâmico do PHP-FPM (CODEIGNITER FOUNDATION, 2024).

3.3 O Ecossistema FrankenPHP e a Arquitetura Worker

O FrankenPHP surge como um servidor de aplicação moderno para PHP, construído sobre o servidor web Caddy em Go (DUNGALAS, 2024; CADDY SERVER, 2024). O diferencial do FrankenPHP reside na integração direta do interpretador PHP (libphp) com o servidor web via threads em Go, eliminando a dependência do protocolo FastCGI (DUNGALAS, 2024).

O FrankenPHP opera sob dois modos principais de execução (DUNGALAS, 2024):

- Modo Clássico: Funciona como um substituto direto e transparente para o PHP-FPM ou Apache/mod_php, executando scripts individualmente a cada requisição (DUNGALAS, 2024).
- Modo Worker: Carrega a aplicação uma única vez na memória durante o processamento inicial e trata as requisições recebidas em um loop contínuo de alta velocidade (DUNGALAS, 2024).

Enquanto o suporte ao Modo Worker é nativo no Laravel por meio do Octane, sua aplicação em ecossistemas como o CodeIgniter 4 exige a implementação de soluções de integração (bridges) customizadas para resetar estados e adaptar o ciclo de vida das requisições (DUNGALAS, 2024; CODEIGNITER FOUNDATION, 2024).

3.4 Avaliação de Desempenho e Benchmarking de Sistemas Web

A análise de desempenho em Engenharia de Software requer uma metodologia rigorosa de medição empírica e controle do ambiente experimental (JAIN, 1991). Segundo Jain (1991), a caracterização de desempenho deve considerar métricas fundamentais como a vazão (throughput), medida em requisições por segundo (RPS), e a latência, que compreende o tempo total de resposta sob diferentes níveis de estresse.

Para submeter os ambientes a condições controladas de carga, utilizam-se ferramentas especializadas de load testing como o Grafana k6 (GRAFANA LABS, 2024). A execução de testes de estresse com perfis de usuários virtuais (VUs) e patamares sustentados permite avaliar não apenas o pico de capacidade do sistema, mas também o consumo de recursos computacionais (processamento de CPU e alocação de memória RAM) em cada arquitetura testada (GRAFANA LABS, 2024; JAIN, 1991).

4 METODOLOGIA

4.1 Caracterização da pesquisa

A pesquisa será caracterizada como experimental, quantitativa e comparativa, tendo como objeto de estudo o desempenho de diferentes arquiteturas de execução para aplicações desenvolvidas com o framework CodeIgniter 4. O caráter experimental decorrerá da execução de testes controlados em diferentes configurações de servidores de aplicação, enquanto a abordagem quantitativa será empregada na coleta e análise de métricas objetivas de desempenho e utilização de recursos computacionais.

A pesquisa terá como foco a comparação entre o modelo tradicional de execução baseado em PHP-FPM e as arquiteturas proporcionadas pelo FrankenPHP, considerando seus modos Clássico e Worker. Adicionalmente, será utilizado o Laravel Octane como cenário de referência, permitindo comparar os resultados obtidos no CodeIgniter 4 com uma solução consolidada no ecossistema PHP para execução de aplicações em servidores de longa duração.

4.2 Ambiente experimental

Para garantir condições controladas e favorecer a reprodutibilidade dos experimentos, os cenários serão executados utilizando contêineres Docker. Os recursos computacionais destinados às aplicações serão limitados de maneira controlada, buscando manter condições equivalentes entre os cenários avaliados.

Serão utilizados os seguintes cenários experimentais:

- Cenário A — PHP-FPM: CodeIgniter 4 executado sobre PHP-FPM, utilizando Nginx como servidor web;
- Cenário B — FrankenPHP Clássico: CodeIgniter 4 executado diretamente sobre o FrankenPHP em seu modo Clássico;
- Cenário C — FrankenPHP Worker: CodeIgniter 4 executado sobre o FrankenPHP em modo Worker, utilizando uma solução de integração que permita manter a aplicação carregada em memória;

- Cenário D — Laravel Octane: aplicação equivalente desenvolvida em Laravel e executada utilizando Laravel Octane, utilizada como referência para os resultados obtidos.

Cada cenário destinado à aplicação será configurado com aproximadamente 1 CPU e 512 MB de memória RAM, buscando estabelecer condições equivalentes de recursos entre as diferentes arquiteturas. O injetor de carga será executado em contêiner separado, evitando que o processamento responsável pela geração das requisições interfira diretamente nos recursos disponíveis para os servidores avaliados.

A aplicação utilizada nos experimentos será desenvolvida com CodeIgniter 4, versão 4.7.3, utilizando PHP 8.3.20. O banco de dados utilizado será o PostgreSQL 16, executado em contêiner separado.

4.3 Instrumentação e geração de carga

A geração de carga será realizada por meio do Grafana k6, executado de forma isolada do ambiente das aplicações. Os testes serão realizados com usuários virtuais (VUs), permitindo submeter cada cenário a uma carga controlada e equivalente.

Cada execução terá duração total de 15 minutos, distribuída da seguinte maneira:

- 2 minutos de aquecimento, com 10 usuários virtuais;
- 2 minutos de rampa, elevando a carga de 10 para 100 usuários virtuais;
- 10 minutos de carga sustentada, com 100 usuários virtuais;
- 1 minuto de desaceleração, reduzindo a carga até zero.

Antes da execução de cada teste, serão adotados procedimentos destinados a reduzir a influência do experimento anterior, incluindo a limpeza do cache do sistema operacional. Dessa forma, busca-se minimizar fatores externos capazes de interferir na comparação entre os cenários.

4.4 Variáveis e métricas analisadas

O desempenho das arquiteturas será avaliado principalmente por meio das seguintes métricas:

- Vazão (RPS): quantidade de requisições processadas por segundo pelo sistema;
- Latência: tempo necessário para processamento e resposta das requisições;
- Utilização de CPU: percentual de utilização dos recursos computacionais disponíveis;
- Utilização de memória RAM: quantidade de memória utilizada durante a execução dos testes.

Os resultados obtidos serão registrados e posteriormente organizados para permitir a comparação entre os cenários. A análise buscará identificar diferenças de desempenho entre o modelo tradicional baseado em PHP-FPM, o FrankenPHP em modo Clássico, o FrankenPHP em modo Worker e o Laravel Octane.

4.5 Procedimentos de análise

Após a realização dos testes, os dados coletados serão organizados e comparados entre os diferentes cenários experimentais. A análise considerará a relação entre vazão, latência e utilização de recursos computacionais.

Serão identificados os cenários que apresentarem maior capacidade de processamento, menor latência e melhor relação entre desempenho e recursos utilizados. Também será considerada a complexidade técnica necessária para utilização do FrankenPHP em modo Worker com o CodeIgniter 4, uma vez que o framework não apresenta suporte nativo a esse modelo de execução.

A partir da comparação dos resultados, será possível avaliar se a utilização de arquiteturas baseadas em persistência de memória apresenta ganhos de desempenho suficientemente relevantes para justificar sua adoção em aplicações desenvolvidas com CodeIgniter 4.

5 CRONOGRAMA

O cronograma apresentado considera que parte significativa do referencial teórico já foi pesquisada, a arquitetura dos contêineres Docker já foi estruturada e implementada, e o benchmark encontra-se em sua etapa final de execução. Dessa forma, as etapas previstas concentram-se principalmente na consolidação do referencial teórico, organização e análise dos dados obtidos, discussão dos resultados e elaboração do trabalho final.

Etapas	Ago	Set	Out	Nov	Dez
Revisão e delimitação do problema de pesquisa	●				
Levantamento bibliográfico	●	●			
Elaboração do Projeto de Pesquisa	●	●			
Revisão do referencial teórico		●	●		
Organização e validação dos dados do benchmark	●	●			
Análise estatística/comparativa dos resultados		●	●		
Redação da metodologia		●	●		
Redação dos resultados e discussão			●	●	
Revisão e adequação às normas ABNT				●	
Redação final do TCC				●	●
Revisão com orientador(a)	●	●	●	●	●
Entrega/apresentação					●

REFERÊNCIAS

CADDY SERVER. **Caddy 2 Documentation**. Light Code Labs, 2024. Disponível em: https://caddyserver.com/docs/. Acesso em: 11 ago. 2026.

CODEIGNITER FOUNDATION. **CodeIgniter 4 User Guide**. 2024. Disponível em: https://codeigniter.com/user_guide/. Acesso em: 11 ago. 2026.

DUNGALAS, Kévin. **FrankenPHP: The Modern Application Server for PHP**. Les-Tilleuls.coop, 2024. Disponível em: https://frankenphp.dev/docs/. Acesso em: 11 ago. 2026.

GRAFANA LABS. **k6 Documentation: Load Testing for Engineering Teams**. 2024. Disponível em: https://grafana.com/docs/k6/. Acesso em: 11 ago. 2026.

JAIN, Raj. **The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling**. New York: Wiley, 1991.

LOCKHART, Josh. PHP Moderno: **Recursos atualizados para uma boa prática de desenvolvimento**. São Paulo: Novatec, 2015.

OTWELL, Taylor. **Laravel Octane Documentation**. Laravel LLC, 2024. Disponível em: https://laravel.com/docs/octane. Acesso em: 11 ago. 2026.

PHP DOCUMENTATION GROUP. PHP: **FastCGI Process Manager (FPM)**. 2024. Disponível em: https://www.php.net/manual/pt_BR/install.fpm.php. Acesso em: 11 ago. 2026.